CEGAS Technical Report

GENERAL IMPLEMENTATION GUIDELINES

> *The Registrar's Office needs a working Campus Enrolment & Grade Analytics System for a pilot section. Staff should be able to manage Students/Teachers/Courses/Sections/Enrolments/Assessments/Scores; run reports (class lists, teacher load, top performers, at-risk students); export CSVs and charts; and recover from a logical backup.*

Our school is still relatively small and is only recently investing in a digital Learning Management System (LMS) for staff, not even for students. Due to resource constrictions, UI/UX design should be avoided for now.

As a cheaper alternative, we will teach staff to use the CEGAS through a Command Line Interface (CLI) directly connected to the database server. This will serve as the only user interface to the database. The CEGAS is for staff use only and does not need a connection to the internet. Without the internet, there is virtually no risk of data leaks other than physical theft. The tech team and administration only shall be given permission to import/export server files, and anyone else needs permission from them before doing so. Security will be enforced by the security department through CCTVs. Finally, password protection via Full Disk Encryption (FDE) will be used on backups and the server to reduce personnel with full access to the database. Thus in general, data protection will be done mostly outside our code.

As only one person can use the server at once, implementing concurrency is not needed. System requirements are also relatively low due to this (see `reports/physical_design.md`). Furthermore, due to zero concurrency, high performance is not a major concern. There is some leeway for suboptimal code, but doing so should be used sparingly for the longevity of the server's hardware.

MySQL will be used minimally to easily display schemas and seeds in code-form, but does not, and will not, be executed. CSV will instead be used to store values.

***CONTEXT NOTE**: My computer requires root access to run MySQL, the query language taught to us. It was agonizing to set up on my system, so I decided to add details to the vocational scenario. I kept it realistic to the best of my abilities and I guarantee that my implementation has all the features expected in any relational database, albeit almost fully in Python and CSV. My database still implements a query language (MySQL) as shown in the last paragraph (04-P3) and adheres to weekly rush requirements as much as possible.*

USED PROGRAMMING PARADIGMS

According to Loyola Marymount University (2019), a *programming paradigm* is a "style," or way, of programming. Three programming paradigms used in the CEGAS code are the following: procedural programming, object-oriented programming, and event-driven programming.

Procedural Programming

Procedural programming is a paradigm in which code can be modularized into smaller chunks, and run at any future point in the code (Ali, 2024). Distinguishing features of procedural programming are libraries and functions.

Procedural programming languages have "functions," which contain an abstracted block of code. The code in a function can be run by "calling" it. Functions can take specified parameters enclosed in parentheses. In Python and many other languages, it would be written as so: `<function_name>(<parameters>).` Functions can return any value(s), as long as the value(s) are within a defined data type.

The CEGAS uses many functions. For example, the function `is_email_format()` runs a block of code. The block of code runs a regex search of the string parameter, and will return a `bool` datatype corresponding to whether it is a valid email address.

Object-Oriented Programming

According to GeeksforGeeks (2025), Object-oriented programming (OOP) is "a programming paradigm based on objects, which can contain data and code." Distinguishing features of OOP are classes, methods, and objects.

OOP contains classes, blueprints of objects. They are similar to structs, except a class can contain methods (functions in a class) and inheritance/polymorphism (a class can be based on another class). This can be useful if you want a simple way to encapsulate and manipulate a data structure without repetition. One can say that OOP implements procedural programming since it implements functions as methods, which are simply functions within a class.

The CEGAS code has class `CEGASCache`, which contains the entire cache of the system. It gives a way to edit, add, and fetch data using simple one-liners.

Event-driven Programming

According to GeeksforGeeks (2024), "[e]vent-driven programming is a paradigm where the execution of a program is determined by events such as user actions or messages." A common way input can be processed is through infinite loops, where user actions are registered repeatedly for the entirety of the code's runtime.

Graphical User Interfaces (GUIs) are an easy way for event-driven programs to handle input, wherein an event can be run through the press of a button. The CEGAS, even without a GUI, can process event-driven programming through an infinite loop wherein the user is prompted to enter an action via `input()` through every loop.

SELECTED PROGRAMMING ENVIRONMENT

All code, Python or MySQL, will be written in Visual Studio Code (VSCode), a text editor made by Microsoft. While not an IDE by default, it is functionally an IDE through extensions, but the ability to choose one's extensions allows more flexibility and saves storage space.

By default, VSCode formats code according to the official Python coding style (van Rossum, Warsaw and Coghlan, 2001). Furthermore, VSCode comes packed with syntax highlighting, useful hotkeys, and autocompletion choices. But most importantly, it highlights and describes potentially-unnecessary code, as well as possible errors via the problems menu (accessible via `ctrl + shift + m`), which helps in the **debugging process** (more on that below).

THE DEBUGGING PROCESS

The debugging process is the step in which the programmer finds and corrects *errors* to ensure that code runs as it was intended. Here are the general steps of debugging: Reproduce, find, determine the cause, fix, re-test, and document (Quiroz-Vázquez, 2024). Practically however, the process heavily depends on the type of error and the programming language.

In the case of Python, there are three main errors (Khan, 2023). These three have different causes, issues, and typical solutions.

Syntax Errors

Syntax errors come from code that does not follow Python's syntactical rules. The Python interpreter cannot parse this, and thus does not run the program. In the example below, the parenthetical is not closed, is not accepted by the Python interpreter, and thus leads to a syntax error.

```
1   print(3

PROBLEMS 6   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   QUERY RESULTS
print(3 2
  "(" was not closed Pylance [Ln 1, Col 6]
  Statements must be separated by newlines or semicolons Pylance [Ln 1, Col 8]
```

Furthermore, you can see that VSCode extensions quickly catch these errors and prompt the user to fix them.

Runtime Errors

Runtime errors allow code to run, until it reaches the error-evoking line of code, immediately killing the program and printing an error stacktrace log to stdout. This default behavior of ending the program can be altered through try- except blocks.

This error is easier to miss since they do not always get detected by IDEs. To solve this problem, debugging breakpoints are commonly used to inspect the current status of the code.

```
1   print('1' + 1)

Exception has occurred: TypeError ×
can only concatenate str (not "int") to str
  File "/home/school/school-stuff/Programming/pearson/example.py", line 1, in <module>
    print('1' + 1)
          ~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

TypeError, a common runtime error due to Python weak typing

Logic Errors

Logic errors have working code, but the logic in the code is faulty, which leads to unexpected or unwanted behavior. This kind of error is harder to diagnose. It could even take a full inspection of the code. These can take hours or even *days* to fix. See the example below.

```
# code runs
expenses = 100
earned = 1000
profit = 100 + 1000
print(profit)
```

In my simplified example, the code runs, prints 1100, and exits successfully. No errors have been highlighted by VSCode. However, the fact of the matter is, profit is not calculated by *adding* expenses to income. The output is faulty, and therefore it is an error.

Synthesis

All of these errors mean broken code, no exceptions. At best, it leads to un-executable code or unexpected behavior. At worst, it becomes a major cybersecurity threat (Aquasec, 2023).

Without the debugging process, attempting to fix these bugs could lead to patching symptoms, and not finding the root, which can cause verbose code and persistent issues. Since the debugging process includes step "determining the cause," it leads to a higher chance of fully resolving the problem, and if that fails, future debuggers have a reference to see what has already been tried.

In my case, I first searched for runtime errors by using test cases on all options in the main loop. I then go through the entire general debugging process to fix this. Once everything runs smoothly without errors, I validate if outputs are accurate and clean to test for logic errors. Of course, I run through the debugging process again with logic errors, however searching is more intensive due to the inherent nature of logic errors.

I estimate that I spent half my time debugging logic errors, and I have almost never run code perfectly without runtime errors, during the first test cases, *especially* ValueErrors and TypeErrors. But thankfully, syntax errors are handled by the IDE and are very easy to spot. Bugs are very common, especially with beginner programmers, and sending broken code is never acceptable.

FINAL TESTING

To test the finished product, everything in the main loop that requires an input will be tested using the test case list (Appendix B, Table A). The list ensures that constraints and logical anomalies (i.e. email address formatting) will be strict such that no anomalous data will be

accepted. Many logic errors were found, noted via color codes, and corrected with the debugging process (Appendix B, Table A).

Aside from the inherent constraints in `varchar(128)`, names were intentionally not filtered strictly because symbols, numbers, and signs, while uncommon, could possibly be part of a name. Different countries have different laws, and not acknowledging this could be potentially controversial.

Additionally, miscellaneous bugs were also tested such as running the program from a different working directory and deleting the `logs/` folder. These were found to work from the first try.

ALGORITHMS

The Merriam-Webster dictionary defines an algorithm as, "a procedure for solving a mathematical problem [...] in a finite number of steps that frequently involves repetition of an operation." This is similar to its application in computer science using code, typically through loops. Common examples are using algorithms to sort, to search for, to create, and to manipulate data.

The CEGAS frequently uses the default `sorted()` function in Python, which uses the *powersort* algorithm as of Python3.11 (James, 2022). *Powersort* is a fork of *Timsort*, the previous default sorting algorithm for Python. Despite its name, it is simply an implementation of *mergesort* simply modified mathematically to be nearly-optimal in most scenarios (Munro and Wild, 2018).

In `io_csv.py`, the `edit_row()` function, though sub-optimal, uses an algorithm in `O(n)` time to insert the appropriate row by finding its primary key in the csv file. It takes two lines and checks if the primary key (sorted in ascending order) fits between two lines. However, it is inefficient in terms of *memory* as it loads the whole file in memory, closes it, and opens it in write mode to rewrite the file line-by-line. This roundabout way is due to the fact that inserting a line *in the middle* of a file is not an option in *any* file mode in Python (Python.org, 2025).

BACKUPS

Logical backups can be easily made through `backup/backup.sh`. It is a bash file that copies `db/csv/*.csv` into the `backup/` directory, which is where all backups are designed to be stored. From then, administrators can choose to zip/export it to store the backup properly.

Backups of the CEGAS program, on the other hand, should already be stored physically and in the cloud before implementing the program for use.

CONCURRENCY NOTE*: The program may experience unexpected behavior if backups are being restored while the program is running.*

APPENDICES

APPENDIX A: Database Seed Character Size Benchmarks

From `app/db/csv/benchmark.log` (run on `seed.sql` default data)

```
filename       chars/lines    avg
————————————————————————————————————
Assessment.csv 772 of 17      45.41
Enrollment.csv 559 of 32      17.47
Section.csv    43 of 4             10.75
Course.csv          254 of 7            36.29
Score.csv      17159 of 451        38.05
Teacher.csv    467 of 7            66.71
Student.csv    2119 of 32          66.22
———————————
Total               21373 of 550   38.86
```

Using `seed.sql`'s generic data, this can predict the average line length of tables holding no null attributes.

APPENDIX B: Test Cases

Table 1 - Main Loop Test Attribute Inputs:

Prompted attributes	Test 1	Test 2	Test 3
int [FK]	2.3 (float)	asdf (str)	3 (invalid FK)
* [NOT NULL]	(null)		
varchar(128)	aaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaa aaaaaaaaaaaaaaaaaaaa aaaaaaaaaaa (exceeds limit)	John (unstripped spaces)	8 (int)
*email varchar(128)	example@example@example.com (faulty email address)	example (not an email address)	example@example. (invalid domain name)
float	123 (int)	Fasdf (str)	555555555555555555555555555555555555 (very high)

LEGEND (first final testing):

- **Green**: Handled perfectly
- **Yellow**: Non-fatal error/logic error
- **Red**: Fatal error

All prompts in the main loop were tested with the above values.

RESOURCES

ALI, H. (2024) *Baeldung*. [Online] Baeldung on Computer Science. Available from : https://www.baeldung.com/cs/procedural-programming.

AQUASEC (2023) *Python Security: 6 Common Risks & What You Can Do About Them*. [Online] Aqua. Available from : https://www.aquasec.com/cloud-native-academy/application-security/python-security/.

DEBIAN (2024) *3.4. Meeting Minimum Hardware Requirements*. [Online] Debian.org. Available from : https://www.debian.org/releases/bookworm/armel/ch03s04.en.html [Accessed 04/12/25].

GEEKSFORGEEKS (2025) *Object Oriented Programming(OOP) Tutorial*. [Online] GeeksforGeeks. Available from : https://www.geeksforgeeks.org/interview-prep/object-oriented-programming-oop-tutorial/ .

GEEKSFORGEEKS (2024) *What Is the Event Driven Programming Paradigm ?* [Online] GeeksforGeeks. Available from : https://www.geeksforgeeks.org/javascript/what-is-the-event-driven-programming-paradigm/.

JAMES, MI. (2022) *Python Now Uses Powersort*. [Online] www.i-programmer.info. Available from : https://www.i-programmer.info/news/216-python/15954-python-now-uses-powersort.html.

KHAN, S. (2023) *Khan Academy*. [Online] Khanacademy.org. Available from : https://www.khanacademy.org/computing/tesda-computational-thinking/xed777e2cccd04061:introduction/xed777e2cccd04061:program-execution/a/errors [Accessed 27/11/25].

LOYOLA MARYMOUNT UNIVERSITY (2019) *Paradigms*. [Online] Lmu.edu. Available from : https://cs.lmu.edu/~ray/notes/paradigms/.

MUNRO, J. and WILD, S. (2018) Nearly-Optimal Mergesorts: Fast, Practical Sorting Methods That Optimally Adapt to Existing Runs. [Online] Available from: doi.org/https://doi.org/10.4230/LIPIcs.ESA.2018.63 [Accessed 19/10/2024].

PYTHON.ORG (2025) *Built-in Functions#open*. [Online] Python Documentation. Available from : https://docs.python.org/3/library/functions.html#open.

QUIROZ-VÁZQUEZ, C. (2024) *Debugging*. [Online] Ibm.com. Available from : https://www.ibm.com/think/topics/debugging [Accessed 27/11/25].

VAN ROSSUM, G., WARSAW, B. and COGHLAN, A. (2001) *PEP 8 – Style Guide for Python Code*. [Online] peps.python.org. Available from : https://peps.python.org/pep-0008/ [Accessed 27/11/25].

Why You Need to Plan Regular Computer Replacements. (2023) [Online] Itfoundations.com. Available from : https://itfoundations.com/blog/why-you-need-to-plan-regular-computer-replacements [Accessed 30/11/25].